

Recursion



Good

Morning

Today's content

- Recursion Introduction
- Factorial
- Fibonacci series
- Power function
- TC & SC of recursive function

Recursion $\rightarrow$ Function calling itself

$\rightarrow$ Breaking the problem into smaller problem to get my answer

$$\text{sum}(5) = \underbrace{1 + 2 + 3 + 4}_{\text{sum}(4)} + 5$$

$$\text{sum}(4) = 1 + 2 + 3 + 4$$

$$\text{sum}(5) = \text{sum}(4) + 5$$

$$\text{sum}(N) = \text{sum}(N-1) + N$$

* Steps to Recursive code

01. Assumption $\rightarrow$ Define what function has to do

02. Main logic $\rightarrow$ Break down the problem into sub problem & generate answer for bigger problem

03. Base condition $\rightarrow$ Condition where we want to stop our recursion

$\rightarrow$ Last valid input for our recursion

Q1. Factorial of a no. ($n \geq 1$)

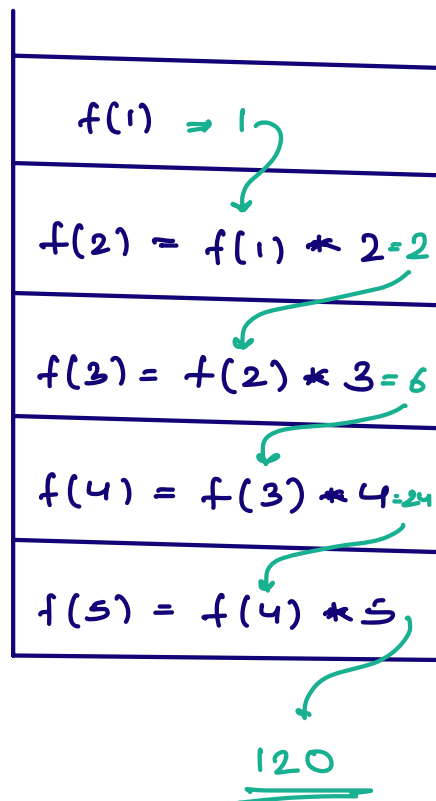
$$\text{Ans of } 5! = 5 * \underbrace{4 * 3 * 2 * 1}_{4!} = 120$$

$$4! = 4 * 3 * 2 * 1$$

Assumption $\rightarrow$ (a) Calculate & return factorial of a no.

Main logic $= \text{fact}(N) = N * \text{fact}(N-1)$

Base case $\Rightarrow$ if ($n == 1$) return 1;



```

int fact (int n)
|
|   if (n==1) return 1;
|   return fact(n-1)*n
3

```

O2 Factorial of a no. ($n \geq 0$)

```

int fact (int n)
|
|   return n == 0 ? 1 : fact(n-1)*n
3

```

condition
 if it is true
 if condition is false

* Fibonacci series ($n \geq 0$)

$n =$ 0 1 2 3 4 5 6 7 ..

Ans 0 1 1 2 3 5 8 13 ..

Input = 6 , ans = 8

Input = 7 , ans = 13

Assumption $\rightarrow$ Create a function that will take input as N & returns the fibonacci of N

Main logic $\rightarrow$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Base condition

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

$$\text{fib}(0) = \text{fib}(-1) + \text{fib}(-2) \quad \alpha$$

$$\text{fib}(1) = \text{fib}(0) + \text{fib}(-1) \quad \alpha$$

$$\text{fib}(2) = \text{fib}(1) + \text{fib}(0) \quad \checkmark$$

```
int fib(int n){  
    if (n <= 1) return n;  
    int a = fib(n-1)  
    int b = fib(n-2)  
    return a+b;  
}
```

```
int fib(int n){
    if (n ≤ 1) return n;
    int a = fib(n-1)
    int b = fib(n-2)
    return a+b;
}
```

n=3

Ans = 2

```
int fib(int n){
    if (n ≤ 1) return n;
    int a = fib(n-1)
    int b = fib(n-2)
    return a+b;
}
```

n=2

```
int fib(int n){
    if (n ≤ 1) return n;
    int a = fib(n-1)
    int b = fib(n-2)
    return a+b;
}
```

n=1

```
int fib(int n){
    if (n ≤ 1) return n;
    int a = fib(n-1)
    int b = fib(n-2)
    return a+b;
}
```

n=1

```
int fib(int n){
    if (n ≤ 1) return n;
    int a = fib(n-1)
    int b = fib(n-2)
    return a+b;
}
```

n=0

* Power function ($n \geq 0$)

Given a & n , find a^n

$$\begin{array}{l} a = 2 \\ n = 4 \end{array} \Rightarrow 2^4 = 16$$

$$2^4 = \underbrace{2 * 2 * 2}_{2^3} * 2 \Rightarrow 16$$

$$2^3 = 2 * 2 * 2$$

$$2^4 = 2^3 * 2$$

$$a^n = a^{n-1} * a$$

Assumption $\rightarrow$ calculate & return a^n

Main logic = $a^n = a^{n-1} * a$

Base condition = if ($n == 0$) return 1;

```
int pow1 (int a, int n)
|
|   if (n == 0) return 1;
|
|   return pow1(a, n-1) * a;
|
3
```

* Power 2

$$a^4 = a^2 * a^2$$

$$a^{12} = a^6 * a^6$$

$$a^{11} = a^5 * a^5 * a$$

$$a^{13} = a^6 * a^6 * a$$

$$\text{if } n = \text{even} \longrightarrow a^n = a^{n/2} * a^{n/2}$$

$$\text{if } n = \text{odd} \longrightarrow a^n = a^{n/2} * a^{n/2} * a$$

* `int pow2 (int a, int n)`

`if (n == 0) return 1;`

`if (n % 2 == 0) return pow2(a, n/2) * pow2(a, n/2);`

`else return pow2(a, n/2) * pow2(a, n/2) * a;`

`}`

* `int pow3 (int a, int n)`

`if (n == 0) return 1;`

`int x = pow3(a, n/2)`

`if (n % 2 == 0) return x * x;`

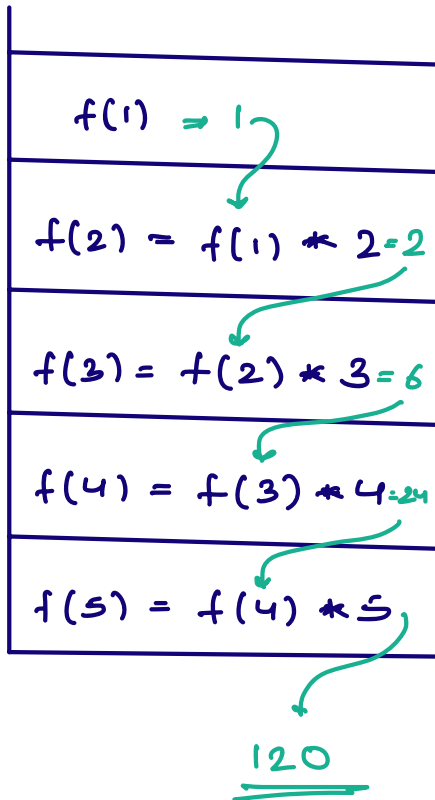
`else return x * x * a;`

`}`

7:56 → 8:06 AM

* Time complexity

01. Factorial function



```
int fact (int n)
{
    if (n==1) return 1;
    return fact(n-1)*n;
}
```

Assumption - $\text{fact}(n)$ will take $T(N)$ time

$$\text{fact}(N-1) = T(N-1) + \text{time}$$

$$T(N) = T(N-1) + 1 \rightarrow 1$$

$$T(N-1) = T(N-2) + 1$$

$$T(N) = T(N-2) + 2 \rightarrow 2$$

$$T(N-2) = T(N-3) + 1$$

$$T(N) = T(N-3) + 3 \rightarrow 3$$

After K iterations

$$T(N) = T(N-K) + K$$

$$N - K = 0$$

$$N = K$$

$$T(N) = 0 + N$$

$$TC = O(N)$$

$$SC = O(N)$$

* Pow1

```
int pow1 (int a, int n)
|
|   if (n == 0) return 1;
|
|   return pow1(a, n-1) * a;
|
3
```

Assumption = $\text{pow1}(a, n)$ will take $T(N)$ time

$\text{pow1}(a, n-1)$ will take $T(N-1)$ time

$$T(N) = T(N-1) + 1$$

$$T(0) = 1$$

$$TC: O(N)$$

$$SC: O(N)$$

```
* int pow2 (int a, int n)
```

```
    if (n == 0) return 1;
```

```
    if (n % 2 == 0) return pow2(a, n/2) * pow2(a, n/2);
```

```
    else return pow2(a, n/2) * pow2(a, n/2) * a;
```

```
}
```

Ass $\rightarrow$ $\text{pow2}(a, n)$ will take $T(N)$ time

$\text{pow2}(a, n/2)$ will take $T(N/2)$ time

$$T(N) = T(N/2) + T(N/2) + 1$$

$$T(N) = 2T(N/2) + 1 \rightarrow \text{I}^x \Rightarrow 2^x T\left(\frac{N}{2^x}\right) + 2^x - 1$$

$$T(N/2) = 2T(N/4) + 1$$

$$T(N) = 2 * (2T(N/4) + 1) + 1$$

$$= 4T(N/4) + 3 \quad \text{---} \quad 2^x = 2^2 T\left(\frac{N}{2^2}\right) + \underbrace{2^2 - 1}$$

$$T(N/4) = 2T(N/8) + 1$$

$$T(N) = 4 * (2T(N/8) + 1) + 3$$

$$= 8T(N/8) + 7 \rightarrow 3^{\text{rd}} \text{ equation}$$

$$\hookrightarrow 2^3 T\left(\frac{N}{2^3}\right) + 2^3 - 1$$

After k^{th} iteration

$$T(N) = 2^k T\left(\frac{N}{2^k}\right) + 2^k - 1$$

$$\frac{N}{2^k} = 0 \Rightarrow N = 0$$

$$T(1) = 1$$

$$\frac{N}{2^k} = 1$$

$$N = 2^k$$

$$k = \log_2 N$$

$$T(N) = N * 1 + N - 1$$

$$= N + N - 1$$

$$= 2N$$

$$TC: O(N)$$

$$SC: O(\log N)$$

pow(2,0)
pow(2,0)
pow(2,1)
pow(2,0)
pow(2,0)
pow(2,1) LR
pow(2,2) LR
pow(2,4)
pow(2,8)

$\text{pow2}(2, 8)$

$\text{pow2}(2, 4)$

$\text{pow2}(2, 2)$

$\text{pow}(2, 2)$

$\text{Pow2}(2, 1)$

$\text{Pow2}(2, 1)$

$\text{pow}(2, 1)$

$\text{pow}(2, 1)$

* $\text{Pow3}(a, n)$

```
int pow3 (int a, int n)
```

```
    if (n == 0) return 1;
```

```
    int x =  $\text{pow3}(a, n/2)$ 
```

```
    if (n % 2 == 0) return x * x;
```

```
    else return x * x * a;
```

```
}
```

$\text{Pow3}(a, n)$ will take $f(n)$

$\text{Pow3}(a, n/2) = f(n/2)$

$$f(n) = f(n/2) + 1 \rightarrow 1^{\text{st}} \text{ equation}$$

$$f(n/2) = f(n/4) + 1$$

$$f(n) = f(n/4) + 2 \rightarrow 2^{\text{nd}} \text{ eqn}$$

$$f(n/4) = f(n/8) + 1$$

$$f(n) = f(n/8) + 3 \rightarrow 3^{\text{rd}} \text{ eqn}$$

After k^{th} iteration

$$f(n) = f(n/2^k) + k$$

$$f(1) = 1$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log n$$

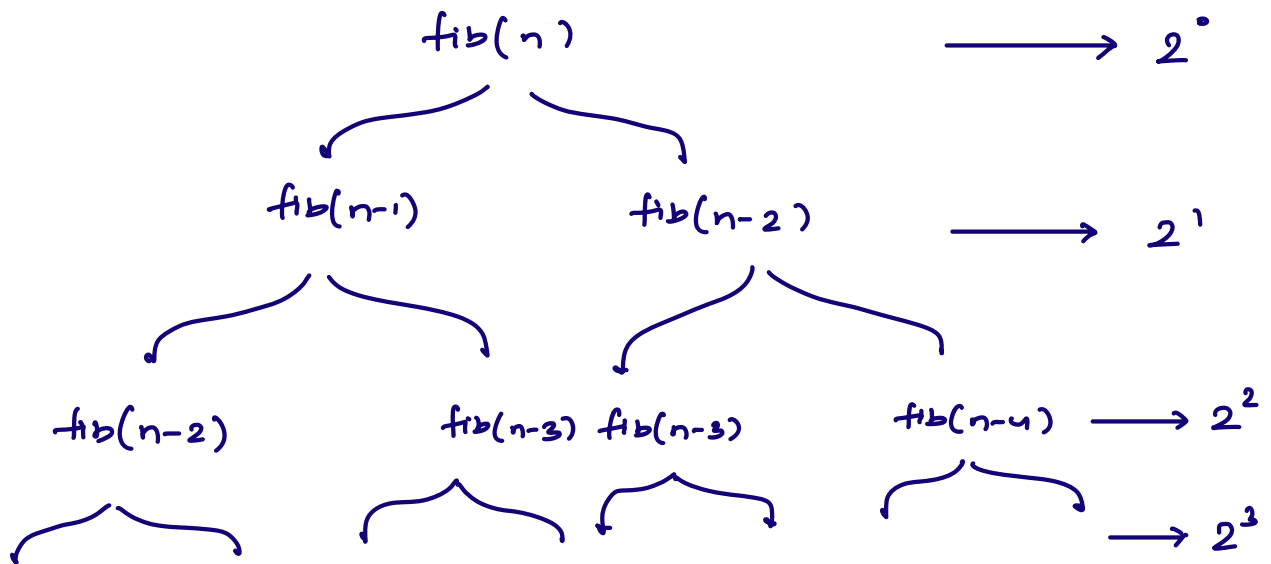
$$f(n) = 1 + \log n$$

$$TC = O(\log n)$$

$$SC = O(\log n)$$

* Fibonacci series

```
int fib(int n){  
    if (n ≤ 1) return n;  
    int a = fib(n-1)  
    int b = fib(n-2)  
    return a+b;  
}
```



$$\text{Total calls} = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^n$$

$$a = 2^0$$

$$r = 2$$

$$\text{no. of terms} = n+1$$

terms

$$\begin{aligned} \text{sum} &= \frac{a(r^n - 1)}{r - 1} = \frac{1(2^{n+1} - 1)}{2 - 1} \\ &= 2^{n+1} - 1 \end{aligned}$$

$$TC = O(2^n)$$

$$SC = O(n)$$

Doubts

* Find two missing no.

$$\underbrace{(1, n+2)}$$

$$\underline{n=4}$$

$$[1, 3, 4, 6] \wedge \underbrace{[1, 2, 3, 4, 5, 6]}$$

$$2 \wedge 5$$

Find two distinct no. from duplicate no.

$$\begin{array}{ccc} \underbrace{[100-91]} & \underbrace{[90-81]} & \underbrace{[80-71]} \\ \text{excellent} & \text{good} & \text{worse} \end{array}$$

if (marks > 71) {

 ==
 ==
 ==
}

(92)